Modelado de Datos con Estructuras y Punteros en C++

Informe Técnico de Laboratorio de Programación — Actividad N.º 5

Ulises Ferrari

Lucas Del Pino

Federico Mattia

Máximo Orué

Escuela de Educación Secundaria Técnica N.º 1 “Eduardo Ader”, Vicente López

Curso: Laboratorio de Programación, 5.º Año 3.º División (Grupos A-B)

Profesores: Ganduglia Yamil y Mansilla Muñoz, York Elías

Fecha: 2026

Resumen

El presente informe expone el desarrollo técnico y teórico correspondiente a la Actividad N.º 5 del espacio curricular Laboratorio de Programación. El objetivo central radicó en la modelación de entidades complejas heterogéneas mediante estructuras de datos (*struct*) en lenguaje C++, así como la manipulación y optimización del acceso a memoria RAM mediante pasaje de parámetros por referencia utilizando punteros y el operador flecha (->). Asimismo, se aborda la integración conceptual de esta estructura lógica como módulo de control para el proyecto integrador anual TeKnical May Cry (TmC) y la resolución de inconvenientes asociados al manejo del buffer de entrada de la consola.

Palabras clave: C++, struct, punteros, operador flecha, buffer de entrada, memoria RAM.

Introducción

En la arquitectura de software desarrollada en lenguajes de bajo y medio nivel como C++, la gestión eficiente de los recursos del sistema es un factor determinante. Durante la creación de sistemas complejos, tales como subsistemas de videojuegos o lectores de hardware, la declaración de variables sueltas genera un código propenso a errores, difícil de mantener y desprovisto de cohesión lógica.

Las estructuras de datos (*struct*) resuelven esta problemática permitiendo agrupar variables de distintos tipos bajo una misma entidad unificada. El presente trabajo documenta la implementación práctica de dicho concepto, evaluando no solo el agrupamiento de datos en bloques contiguos de memoria, sino también el acceso de alto rendimiento mediante punteros directos a la memoria RAM. El repositorio del proyecto se encuentra en https://github.com/maximoorue30-tech/sistemaproyecto.git

Desarrollo

Marco teórico y desarrollo técnico

Estructuras heterogéneas (struct)

Una estructura en C++ es una colección de variables (denominadas miembros) agrupadas en un bloque contiguo de memoria. A diferencia de los arreglos tradicionales, las estructuras son heterogéneas, permitiendo combinar tipos enteros, flotantes y cadenas de caracteres. Dentro del marco del proyecto integrador TeKnical May Cry (TmC), la estructura *EntidadProyecto* fue adaptada para representar el subsistema de monitoreo de energía y estado de los módulos del minijuego:

```
struct EntidadProyecto {
    int id;  // Identificador unico del modulo o camara char
    nombre[50];      // Nombre o descripcion del sector
    float metrica;   // Porcentaje de bateria o consumo de energia
};
```

Pasaje por referencia y operador flecha (->)

Cuando una estructura se pasa a una función por valor, el compilador realiza una copia completa de todos sus miembros en el Stack de la pila de llamadas. Esto incrementa el consumo de memoria y reduce el rendimiento.

Para optimizar este proceso, se utiliza el pasaje por referencia mediante punteros (*EntidadProyecto* ptr*), enviando únicamente la dirección de memoria física en formato hexadecimal. Dado que la variable receptora es una dirección y no el objeto directamente, el acceso a sus campos requiere la sintaxis del operador flecha (*ptr->campo*), equivalente a la desreferenciación explícita *(*ptr).campo*.

Control y limpieza del buffer de entrada (cin.ignore)

Un problema recurrente en el desarrollo en C++ ocurre al alternar la lectura de datos numéricos mediante *cin* >> y cadenas de texto mediante *cin.getline()*. El operador *cin* >> lee el dato introducido pero deja el salto de línea (\n o Enter) en el buffer de entrada de la consola. Si no se limpia dicho buffer, la función *cin.getline()* posterior consume inmediatamente ese Enter remanente, omitiendo la lectura del texto y guardando una cadena vacía. La instrucción *cin.ignore()* soluciona esta anomalía descartando el carácter residual antes de leer la cadena.

Implementación del código fuente

El programa fue desarrollado y probado en el entorno Visual Studio Code utilizando el compilador MinGW (g++) sobre el sistema operativo Windows.

```
#include <iostream>
#include <string>
using namespace std;

struct EntidadProyecto {
    int id;
    char nombre[50];
    float metrica; // Carga de bateria / estado del modulo
};

// Prototipo de funcion con pasaje por direccion mediante puntero
void cargarDatos(EntidadProyecto* ptr);

int main() {
    // Inicializacion de seguridad en la memoria Stack
    EntidadProyecto miEntidad;
    miEntidad.id = 0;
    strcpy(miEntidad.nombre, "Vacio - Maximo Orue");
    miEntidad.metrica = 0.0f;

    cout << "================================================" << endl;
    cout << " MODELADO STRUCT - ESTUDIANTE: Maximo Orue " << endl;
    cout << "================================================" << endl;

    // Invocacion enviando la direccion fisica de memoria con '4'
    cargarDatos(miEntidad);

    cout << "\n=== DATOS VERIFICADOS EN LA MEMORIA RAM ===" << endl;
    cout << "ID Registrado: " << miEntidad.id << endl;
    cout << "Nombre Registrado: " << miEntidad.nombre << endl;
    cout << "Metrica Guardada: " << miEntidad.metrica << endl;
    cout << "Direccion RAM Hexadecimal: " << miEntidad << endl;
    cout << "================================================" << endl;
    return 0; // Finalizacion exitosa
}

void cargarDatos(EntidadProyecto* ptr) {
    cout << "\n-- INGRESO DE DATOS MEDIANTE OPERADOR FLECHA --" << endl;
    cout << "=> Ingrese el ID de la entidad (entero): "; cin
    >> ptr->id;
    cin.ignore();
    cout << "=> Ingrese el Nombre o Descripcion: ";
    cin.getline(ptr->nombre, 50);
    cout << "=> Ingrese la Metrica de Operacion (decimal/float): "; cin
    >> ptr->metrica;
```

Respuestas al cuestionario técnico de control

1. Heterogeneidad

Permite agrupar variables de múltiples tipos de datos (int, char[], float) en un único bloque contiguo de memoria RAM bajo un identificador lógico común. Esto evita la desorganización de declarar variables independientes y simplifica el paso de parámetros a

funciones.

2. Operador flecha (->)

El operador de punto (.) aplica exclusivamente sobre instancias u objetos directos. Cuando se manipula una variable de tipo puntero, esta no contiene los datos sino una dirección de memoria; por ello, el compilador exige usar la sintaxis *ptr->campo* para desreferenciar el puntero y acceder al miembro correspondiente.

3. Eficiencia en RAM

Al pasar una estructura mediante punteros, únicamente se copia la dirección de memoria física (de 4 u 8 bytes) en el Stack. Esto evita duplicar pesadas estructuras de datos en la memoria RAM, acelerando el tiempo de procesamiento de la aplicación.

4. Buffer de entrada

Si se omite la instrucción *cin.ignore()*, el salto de línea (\n) dejado por la lectura previa del entero es leído por *cin.getline()*, provocando que el programa no espere la interacción del usuario e ingrese una cadena vacía automáticamente.

5. Aplicación al Proyecto Integrador (TmC)

Se representó el módulo del Sistema de Monitoreo de Seguridad / Batería perteneciente al minijuego tipo FNaF. Sus miembros son *id* (identificador del módulo), *nombre* (sector de la cámara) y *metrica* (porcentaje de energía restante).

Distribución de roles y tareas del equipo

- **Ulises Ferrari:** investigación conceptual de la normativa APA v7, redacción del documento técnico, resolución del cuestionario y elaboración del manual.
- **Lucas Del Pino:** programación en C++, depuración de errores de punteros, ejecución en terminal y pruebas de compilación.
- **Federico Mattia:** organización de la jerarquía del repositorio en GitHub, maquetado y revisión de formatos.

- **Máximo Orué:** control de calidad integral, verificación del cumplimiento de pautas del docente y actualización de la Planilla de Seguimiento.

Conclusión

La actividad permitió modelar una entidad heterogénea con *struct* y comprender las ventajas de pasarla a una función mediante un puntero: se copia solo la dirección de memoria y no toda la estructura, y el acceso a los miembros se realiza con el operador flecha. También se resolvió el problema del salto de línea residual en el buffer de entrada con *cin.ignore()*. Finalmente, la estructura se adaptó como módulo de monitoreo para el proyecto integrador TeKnical May Cry (TmC), lo que muestra su aplicación en un proyecto real.

Referencias

Cplusplus.com. (2026). *Data structures (struct)*. Documentation - C++ Language. https://cplusplus.com/doc/tutorial/structures/

E.E.S.T. N.º 1 “Eduardo Ader”. (2026). *Guía de Actividad 5: Modelado de datos con estructuras (struct) y punteros* [Consigna de la actividad]. Laboratorio de Programación.

Ferrari, U., Del Pino, L., Mattia, F., y Orué, M. (2026). *sistemaproyecto* [Repositorio de código]. GitHub. https://github.com/maximoorue30-tech/sistemaproyecto.git

Microsoft Learn. (2026). *Pointers and reference parameters in C++*. Microsoft C++ Language Documentation.